



Matematika FMIPA
Universitas Padjadjaran

ANALISIS SISTEM ANTRIAN PEMBAYARAN REGISTRASI MAHASISWA DENGAN MODEL ANTRIAN SINGLE CHANNEL-SINGLE PHASE POLA M/M/1



Kelompok 4



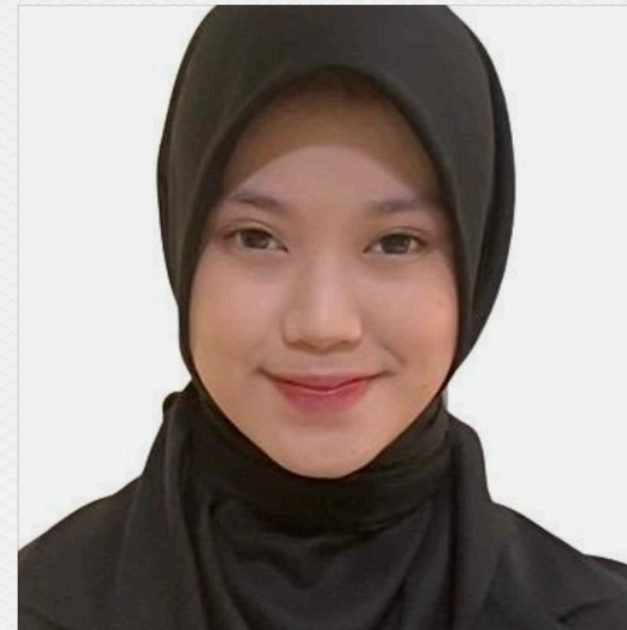
OUR TEAM



LILIS CAHYANI
140110230007



IREN REBECCA
140110230014



NASYA SAFINA
140110230046



DINDA ALYA
140110230059

Latar Belakang

Salah satu penyebab terjadinya antrian adalah ketidakseimbangan antara jumlah pelanggan dan kapasitas pelayanan yang tersedia. Jika laju kedatangan pelanggan lebih besar dibandingkan kemampuan petugas dalam melayani, maka antrian akan menumpuk dan waktu tunggu semakin panjang. Kondisi ini umum terjadi di berbagai sistem pelayanan, termasuk pada sektor perbankan, di mana nasabah sering mengalami antrean panjang pada jam sibuk.

Untuk memahami dan mengoptimalkan kondisi tersebut, diperlukan analisis menggunakan teori antrian, khususnya model $M/M/s$ yang menggambarkan sistem dengan beberapa petugas pelayanan (server). Melalui simulasi menggunakan Python, dapat diketahui performa sistem berdasarkan parameter seperti tingkat kesibukan, panjang antrian, waktu tunggu, serta biaya pelayanan.

Dengan penerapan model ini, proyek ini bertujuan menemukan jumlah server yang paling efisien, sehingga sistem pelayanan dapat berjalan optimal dengan waktu tunggu minimal dan biaya operasional yang seimbang.

Penerapan Model Antrian M/M/s pada Sistem Locket Pembayaran

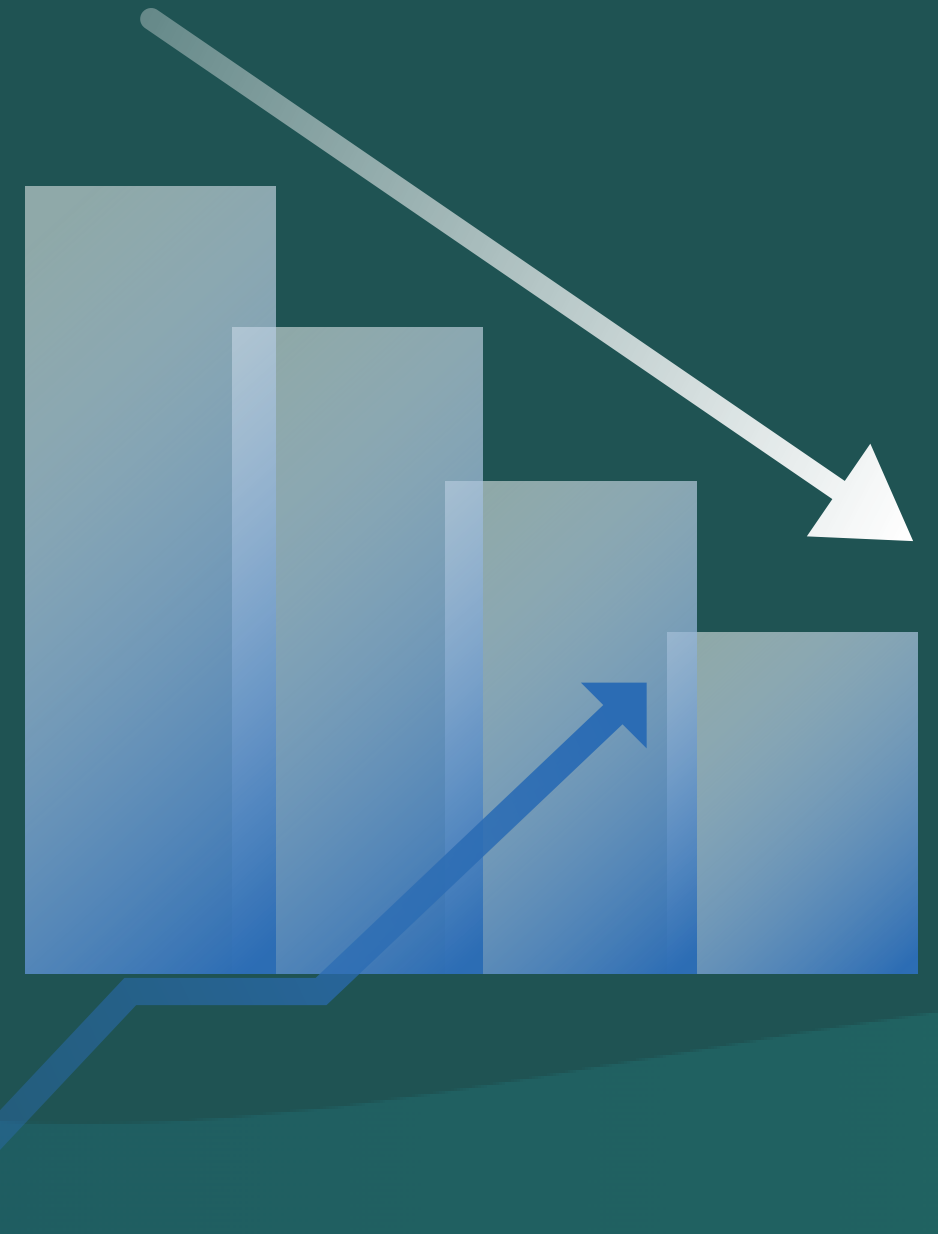
Topik yang diangkat dalam proyek ini adalah analisis sistem antrian model M/M/s (Multiserver) yang diterapkan pada loket pembayaran di Universitas Nusa Nipa (UNIPA). Permasalahan utama yang sering terjadi di loket pembayaran adalah antrian panjang pada jam-jam sibuk, akibat banyaknya mahasiswa yang datang bersamaan sementara jumlah petugas loket terbatas. Kondisi ini menimbulkan waktu tunggu yang lama dan menurunkan efisiensi pelayanan.

Model M/M/s digunakan karena dapat merepresentasikan sistem dengan lebih dari satu petugas (server) dan pola kedatangan mahasiswa yang bersifat acak, yang secara umum mendekati distribusi Poisson. Pendekatan model ini tetap digunakan untuk memperoleh estimasi kinerja sistem seperti tingkat kesibukan (ρ), panjang antrian (L_q), dan waktu tunggu rata-rata (W_q) secara praktis dan terukur.

Untuk mengimplementasikan model tersebut, dibuat fungsi menggunakan bahasa pemrograman Python yang akan menghitung seluruh parameter kinerja antrian serta menambahkan komponen biaya pelayanan dan biaya menunggu untuk menilai efisiensi sistem secara menyeluruh.

Tujuan

Proyek ini bertujuan untuk menganalisis dan mengoptimalkan sistem antrian pada loket pembayaran Universitas Nusa Nipa menggunakan model M/M/s agar pelayanan menjadi lebih efisien. Melalui simulasi berbasis Python, proyek ini diharapkan dapat menentukan jumlah petugas loket yang optimal dengan mempertimbangkan keseimbangan antara waktu tunggu mahasiswa dan biaya pelayanan yang dikeluarkan.



Batasan

1. Proyek ini hanya berfokus pada analisis sistem antrian di loket pembayaran Universitas Nusa Nipa selama periode registrasi mahasiswa bulan Desember 2021.

2. Data yang digunakan meliputi tingkat kedatangan (λ) dan tingkat pelayanan (μ) yang diperoleh dari hasil observasi langsung di lapangan.

3. Model yang digunakan adalah model antrian M/M/s, dengan variasi jumlah server dari $s = 1$ hingga $s = 4$, untuk melihat perbandingan efisiensi pelayanan.

4. Perhitungan dilakukan dengan simulasi Python, mencakup probabilitas sistem sibuk, panjang antrian, waktu tunggu, dan total biaya

5. Proyek ini tidak membahas faktor eksternal, hanya menitikberatkan pada jumlah server, waktu tunggu, dan biaya pelayanan.

Analisis Kasus

dan Implementasi Program

Identifikasi Permasalahan

Tabel 1. Data Kedatangan

Hari	Jumlah Kedatangan (Jam)		
	10—11	11—12	12—13
Hari 1	180	140	60
Hari 2	159	160	80
Hari 3	160	195	50

Tabel 2. Data Pelayanan

Hari	Jumlah Pelayanan (Jam)		
	10—11	11—12	12—13
Hari 1	50	45	30
Hari 2	59	60	40
Hari 3	60	90	60

Berdasarkan hasil pengamatan di loket pembayaran Universitas Nusa Nipa (UNIPA), terlihat bahwa jumlah kedatangan mahasiswa jauh melebihi kapasitas pelayanan yang dapat diselesaikan petugas, terutama pada interval waktu 10.00–11.00 dengan rata-rata lebih dari 160 mahasiswa per jam, sementara kapasitas pelayanan per petugas hanya sekitar 55–60 orang per jam.

Ketimpangan antara laju kedatangan (λ) dan laju pelayanan (μ) menunjukkan bahwa sistem pelayanan mengalami kelebihan beban (overload) yang berdampak pada meningkatnya waktu tunggu dan antrian panjang di loket.

Oleh karena itu, perlu dilakukan simulasi model antrian M/M/s (Multiserver) untuk menentukan jumlah petugas loket yang paling efisien, dengan mempertimbangkan keseimbangan antara tingkat kesibukan sistem (ρ), waktu tunggu rata-rata (Wq), dan biaya pelayanan.

Inisialisasi data dalam DataFrame

```
data_kedatangan = {"Hari": ["Hari 1", "Hari 2", "Hari 3"],
                   "10-11": [180, 159, 160],
                   "11-12": [140, 160, 195],
                   "12-13": [60, 80, 50]}
data_pelayanan = {"Hari": ["Hari 1", "Hari 2", "Hari 3"],
                  "10-11": [50, 59, 60],
                  "11-12": [45, 60, 90],
                  "12-13": [30, 40, 60]}
kedatangan_df = pd.DataFrame(data_kedatangan)
pelayanan_df = pd.DataFrame(data_pelayanan)

print("Data kedatangan:")
display(kedatangan_df)
print("\nData Pelayanan:")
display(pelayanan_df)
```

Data kedatangan:

	Hari	10-11	11-12	12-13
0	Hari 1	180	140	60
1	Hari 2	159	160	80
2	Hari 3	160	195	50

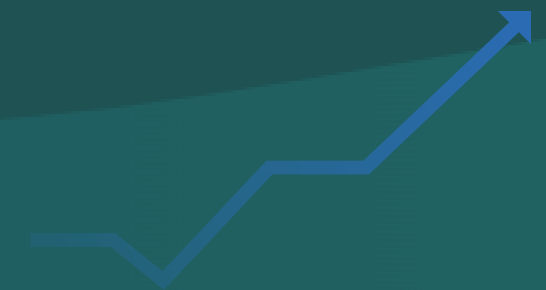
Data Pelayanan:

	Hari	10-11	11-12	12-13
0	Hari 1	50	45	30
1	Hari 2	59	60	40
2	Hari 3	60	90	60

Inisialisasi parameter dasar (λ dan μ)

```
total_kedatangan = kedatangan_df[["10-11", "11-12", "12-13"]].to_numpy().sum()
total_pelayanan = pelayanan_df[["10-11", "11-12", "12-13"]].to_numpy().sum()
n_interval = 9

# Menghitung  $\lambda$  dan  $\mu$ 
 $\lambda$  = total_kedatangan / n_interval
 $\mu$  = total_pelayanan / n_interval
```



Fungsi (M/M/s)

1. Inisialisasi & Validasi

```
[7]: # fungsi (M/M/s)
def hitung_mms(lambd, mu, s):
    if mu <= 0 or lambd <= 0 or s < 1:
        print("λ dan μ harus > 0, dan s harus >= 1")
        return None

    rho = lambd / (s * mu)
    if rho >= 1:
        print(f"ρ = λ/(s μ) = {rho:.4f} ≥ 1 → Sistem tidak stabil (λ harus < sμ)\n")
```

Dilakukan proses inisialisasi fungsi dengan memasukkan parameter λ (laju kedatangan), μ (laju pelayanan), dan s (jumlah server), lalu divalidasi untuk memastikan sistem memenuhi syarat kestabilan sebelum disimulasikan.

2. Rumus Inti M/M/s (Erlang-C)

```
r = lambd / mu

sum_finite = 0.0
for n in range(s):
    sum_finite += (r**n)/math.factorial(n)
tail_term = (r**s)/(math.factorial(s)*(1-rho))

P0 = 1/(sum_finite + tail_term)
Lq = P0*(r**s)*rho/(math.factorial(s)*(1-rho)**2)
L = Lq + lambd/mu
Wq = Lq/lambd
W = Wq + 1/mu
```

Rumus Erlang-C akan menghitung peluang sistem kosong (P_0), panjang antrian (L_q), dan waktu tunggu rata-rata (W_q) agar diperoleh ukuran kinerja sistem secara kuantitatif.

Fungsi (M/M/s)

3. Analisis Biaya Pelayanan

```
Cs = 30000
Cw = 10000
service_cost = Cs * s
waiting_cost = Cw * Lq
total_cost = service_cost + waiting_cost
```

Hitung total biaya sistem dengan menjumlahkan biaya pelayanan (service cost) untuk petugas dan biaya menunggu (waiting cost) akibat antrean, sehingga diperoleh gambaran efisiensi biaya operasional secara keseluruhan.

4. Print Output

```
print("=== HASIL SIMULASI MODEL ANTRIAN M/M/s ===")
print(f"Jumlah server (s): {s_val}")
print(f"Laju kedatangan ( $\lambda$ ): {lambda_val:.2f} pelanggan/jam")
print(f"Laju pelayanan ( $\mu$ ): {mu_val:.2f} pelanggan/jam per server")
print(f"Tingkat kesibukan sistem ( $\rho$ ): {hasil['rho']:.4f}")

print("\n=== Parameter Kinerja Sistem ===")
print(f"Probabilitas tidak ada pelanggan (P0): {hasil['P0']:.4f}")
print(f"Rata-rata pelanggan dalam antrian (Lq): {hasil['Lq']:.4f}")
print(f"Rata-rata pelanggan total dalam sistem (L): {hasil['L']:.4f}")
print(f"Waktu tunggu rata-rata dalam antrian (Wq): {hasil['Wq']*60:.2f} menit")
print(f"Waktu rata-rata dalam sistem (W): {hasil['W']*60:.2f} menit")

print("\n=== ANALISIS BIAYA ===")
print(f"Biaya pelayanan per jam : Rp{hasil['service_cost']:,.0f}")
print(f"Biaya menunggu per jam : Rp{hasil['waiting_cost']:,.0f}")
print(f"Total biaya per jam : Rp{hasil['total_cost']:,.0f}")
```


Fungsi (M/M/s)

4. Print Output

```
# --- interpretasi gabungan efisiensi & waktu tunggu ---
print("\n📊 Interpretasi:")

# efisiensi sistem (berdasarkan rho)
if hasil['rho'] < 0.7:
    print("Sistem tergolong longgar – petugas sering idle, biaya relatif tinggi.")
elif hasil['rho'] < 0.9:
    print("Sistem berada pada kondisi efisien (stabil dan seimbang).")
else:
    print("Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.")

# waktu tunggu (berdasarkan Wq)
if hasil['Wq'] * 60 < 5:
    print("🕒 Waktu tunggu rata-rata sangat baik (<5 menit).")
elif hasil['Wq'] * 60 < 10:
    print("⌚ Waktu tunggu masih tergolong wajar (5-10 menit).")
else:
    print("🕒 Waktu tunggu cukup lama, pertimbangkan penambahan server atau peningkatan kecepatan layanan.")
```

Menampilkan hasil perhitungan model antrian M/M/s secara terstruktur, mencakup parameter kinerja sistem, waktu tunggu, serta analisis biaya untuk memudahkan interpretasi efisiensi layanan dan interpretasi otomatis berdasarkan nilai ρ (tingkat kesibukan sistem) dan W_q (waktu tunggu rata-rata) untuk menilai apakah sistem berada dalam kondisi efisien, longgar, atau overload.





1. Input Parameter dan Pemanggilan fungsi

```
lambda_val = λ  
mu_val = μ  
s_val = 1  
  
hasil = hitung_mms(lambda_val, mu_val, s_val)
```

Karena sistem berada pada kondisi tidak stabil ($\rho \geq 1$), hasil perhitungan menjadi tidak logis, terutama pada nilai rata-rata waktu tunggu dan jumlah pelanggan yang bernilai negatif. Hal ini terjadi karena laju kedatangan jauh melebihi kemampuan pelayanan sistem. Jadi, perhitungan akan dilanjutkan menggunakan pendekatan finite horizon agar tetap dapat memperkirakan kondisi antrian dalam jangka waktu tertentu.

2. Output Model Antrian M/M/1

$\rho = \lambda / (s \mu) = 2.3968 \geq 1 \rightarrow$ Sistem tidak stabil (λ harus $< s\mu$)

=== HASIL SIMULASI MODEL ANTRIAN M/M/1 ===

Jumlah server (s) : 1
Laju kedatangan (λ) : 131.56 pelanggan/jam
Laju pelayanan (μ) : 54.89 pelanggan/jam per server
Tingkat kesibukan sistem (ρ) : 2.3968
⚠ Sistem sangat sibuk, potensi antrean panjang.

=== Parameter Kinerja Sistem ===

Probabilitas tidak ada pelanggan (P_0) : -1.3968
Rata-rata pelanggan dalam antrian (L_q) : -4.1127
Rata-rata pelanggan total dalam sistem (L) : -1.7159
Waktu tunggu rata-rata dalam antrian (W_q) : -1.88 menit
Waktu rata-rata dalam sistem (W) : -0.78 menit

=== ANALISIS BIAYA ===

Biaya pelayanan per jam : Rp30,000
Biaya menunggu per jam : Rp-41,127
Total biaya per jam : Rp-11,127



Interpretasi:

Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.



Waktu tunggu rata-rata sangat baik (<5 menit).

Fungsi (M/M/s)

Dengan Pendekatan Finite Horizon

Jika tidak stabil, akan ada backlog yang tumbuh selama horizon

```
def hitung_mms_finite_horizon(lambd, mu, s, horizon_hours=1.0):
    import math

    if mu <= 0 or lambd <= 0 or s <= 0:
        raise ValueError("λ, μ harus > 0 dan s >= 1")

    rho = lambd / (s*mu)

    # Estimasi finite-horizon bila tidak stabil ( $\rho \geq 1$ )
    if rho >= 1:
        # backlog tumbuh sebesar  $(\lambda - s\mu)$  per jam selama horizon H
        add_backlog = (lambd - s*mu) * max(horizon_hours, 0.0)
        Lq_est = max(0.0, add_backlog)
        Wq_est = Lq_est / lambd
        W_est = Wq_est + 1.0/mu
        L_est = Lq_est + lambd/mu

    Cs, Cw = 30000, 10000
    service_cost = Cs * s
    waiting_cost = Cw * Lq_est
    total_cost = service_cost + waiting_cost
```

```
return {
    'stable': False,
    'rho': rho, 'P0': 0.0,
    'Lq': Lq_est, 'Wq': Wq_est, 'W': W_est, 'L': L_est,
    'service_cost': service_cost,
    'waiting_cost': waiting_cost,
    'total_cost': total_cost,
    'horizon_hours': horizon_hours} #penanda ini angka horizon
```

Jika sistem stabil, gunakan rumus biasa

```
r = lambd / mu
sum_finite = sum((r**n)/math.factorial(n) for n in range(s))
tail_term = (r**s)/(math.factorial(s)*(1 - rho))
P0 = 1.0/(sum_finite + tail_term)

Lq = P0 * (r**s) * rho / (math.factorial(s) * (1 - rho)**2)
Wq = Lq / lambd
W = Wq + 1.0/mu
L = Lq + lambd/mu

Cs, Cw = 30000, 10000
service_cost = Cs * s
waiting_cost = Cw * Lq
total_cost = service_cost + waiting_cost

return {
    'stable': True,
    'rho': rho, 'P0': P0,
    'Lq': Lq, 'Wq': Wq, 'W': W, 'L': L,
    'service_cost': service_cost,
    'waiting_cost': waiting_cost,
    'total_cost': total_cost}
```

Fungsi (M/M/s)



Fungsi untuk menampilkan hasil dengan rapi

Dengan Pendekatan Finite Horizon

```
def hitung_mms_finite_horizon(lambd, mu, s, horizon_hours):
    if mu <= 0 or lambd <= 0 or s <= 0:
        raise ValueError("λ, μ harus > 0 dan s >= 1")

    rho = lambd / (s*mu)

    # Estimasi finite-horizon bila tidak stabil (ρ ≥ 1)
    if rho >= 1:
        # backlog tumbuh sebesar (λ - sμ) per jam selama horizon H
        add_backlog = (lambd - s*mu) * max(horizon_hours, 0.0)
        Lq_est = max(0.0, add_backlog)
        Wq_est = Lq_est / lambd
        W_est = Wq_est + 1.0/mu
        L_est = Lq_est + lambd/mu

    Cs, Cw = 30000, 10000
    service_cost = Cs * s
    waiting_cost = Cw * Lq_est
    total_cost = service_cost + waiting_cost

    return {
        'stable': False,
        'rho': rho, 'P0': 0.0,
        'Lq': Lq_est, 'Wq': Wq_est, 'W': W_est, 'L': L_est,
        'service_cost': service_cost,
        'waiting_cost': waiting_cost,
        'total_cost': total_cost,
        'horizon_hours': horizon_hours} #penanda ini angka horizon
```

```
# efisiensi sistem (berdasarkan rho)
if hasil['rho'] < 0.7:
    print("Sistem tergolong longgar – petugas sering idle, biaya relatif tinggi.")
elif hasil['rho'] < 0.9:
    print("Sistem berada pada kondisi efisien (stabil dan seimbang).")
else:
    print("Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.")

# waktu tunggu (berdasarkan Wq)
if hasil['Wq'] * 60 < 5:
    print("🕒 Waktu tunggu rata-rata sangat baik (<5 menit).")
elif hasil['Wq'] * 60 < 10:
    print("⌚ Waktu tunggu masih tergolong wajar (5-10 menit).")
else:
    print("🕒 Waktu tunggu cukup lama, "
          "pertimbangkan penambahan server atau peningkatan kecepatan layanan.")
```

Simulasi M/M/1

Hasil Simulasi 1 Loker Pembayaran



1. Input Parameter Pemanggilan fungsi (M/M/s) dengan Pendekatan Finite Horizon

```
lambda_val = λ  
mu_val = μ  
s_val = 1  
h_hours = 1  
hasil = hasil_mms_finite_horizon(lambda_val, mu_val, s_val, h_hours)
```

2. Output Model Antrian M/M/1 dengan Pendekatan Finite Horizon

```
=== HASIL SIMULASI MODEL ANTRIAN M/M/1 ===  
Jumlah server (s): 1  
Laju kedatangan (λ): 131.56 pelanggan/jam  
Laju pelayanan (μ): 54.89 pelanggan/jam per server  
Tingkat kesibukan sistem (ρ): 2.3968  
⚠ Sistem tidak stabil (ρ ≥ 1)  
Angka di bawah ini adalah ESTIMASI untuk horizon 1 jam agar bisa dibandingkan:
```

```
=== Parameter Kinerja Sistem ===  
P0: 0.0000  
Lq: 76.6667  
L : 79.0634  
Wq: 34.97 menit  
W : 36.06 menit
```

```
=== ANALISIS BIAYA ===  
💰 Biaya pelayanan per jam : Rp30,000  
📅 Biaya menunggu per jam : Rp766,667  
📄 Total biaya per jam : Rp796,667  
  
📊 Interpretasi:  
Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.  
🕒 Waktu tunggu cukup lama, pertimbangkan penambahan server atau peningkatan kecepatan layanan.
```




1. Input Parameter dan Pemanggilan fungsi

```
lambda_val = λ  
mu_val = μ  
s_val = 2  
  
hasil = hitung_mms(lambda_val, mu_val, s_val)
```

Sama seperti M/M/1, karena sistem berada pada kondisi tidak stabil / overload ($\rho \geq 1$), hasil perhitungan menjadi tidak logis, terutama pada nilai rata-rata waktu tunggu dan jumlah pelanggan yang bernilai negatif. Hal ini terjadi karena laju kedatangan jauh melebihi kemampuan pelayanan sistem. Jadi, perhitungan akan dilanjutkan menggunakan pendekatan finite horizon agar tetap dapat memperkirakan kondisi antrian dalam jangka waktu tertentu.

2. Output Model Antrian M/M/2

$\rho = \lambda / (s \mu) = 1.1984 \geq 1 \rightarrow$ Sistem tidak stabil (λ harus $< s\mu$)

=== HASIL SIMULASI MODEL ANTRIAN M/M/2 ===

Jumlah server (s)	: 2
Laju kedatangan (λ)	: 131.56 pelanggan/jam
Laju pelayanan (μ)	: 54.89 pelanggan/jam per server
Tingkat kesibukan sistem (ρ)	: 1.1984

⚠ Sistem sangat sibuk, potensi antrean panjang.

=== Parameter Kinerja Sistem ===

Probabilitas tidak ada pelanggan (P_0)	: -0.0902
Rata-rata pelanggan dalam antrian (L_q)	: -7.8925
Rata-rata pelanggan total dalam sistem (L)	: -5.4957
Waktu tunggu rata-rata dalam antrian (W_q)	: -3.60 menit
Waktu rata-rata dalam sistem (W)	: -2.51 menit

=== ANALISIS BIAYA ===

Biaya pelayanan per jam	: Rp60,000
Biaya menunggu per jam	: Rp-78,925
Total biaya per jam	: Rp-18,925



Interpretasi:

Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.



Waktu tunggu rata-rata sangat baik (<5 menit).

Simulasi M/M/2

Hasil Simulasi 2 Loker Pembayaran



1. Input Parameter Pemanggilan fungsi (M/M/s) dengan Pendekatan Finite Horizon

```
lambda_val =  $\lambda$ 
mu_val =  $\mu$ 
s_val = 2

hasil = hasil_mms_finite_horizon(lambda_val, mu_val, s_val)
```

2. Output Model Antrian M/M/2 dengan Pendekatan Finite Horizon

```
=== HASIL SIMULASI MODEL ANTRIAN M/M/2 ===
Jumlah server (s): 2
Laju kedatangan ( $\lambda$ ): 131.56 pelanggan/jam
Laju pelayanan ( $\mu$ ): 54.89 pelanggan/jam per server
Tingkat kesibukan sistem ( $\rho$ ): 1.1984
⚠ Sistem tidak stabil ( $\rho \geq 1$ )
Angka di bawah ini adalah ESTIMASI untuk horizon 1 jam agar bisa dibandingkan
```

```
=== Parameter Kinerja Sistem ===
P0: 0.0000
Lq: 21.7778
L : 24.1745
Wq: 9.93 menit
W : 11.03 menit
```

=== ANALISIS BIAYA ===

💰 Biaya pelayanan per jam : Rp60,000
⌚ Biaya menunggu per jam : Rp217,778
📄 Total biaya per jam : Rp277,778



Interpretasi:

Sistem overload – beban pelayanan terlalu tinggi, risiko antrean panjang.
⌚ Waktu tunggu masih tergolong wajar (5-10 menit).

Simulasi M/M/3

Hasil Simulasi 3 Locket Pembayaran



1. Input Parameter & Pemanggilan Fungsi

```
# Jika terdapat 3 loket, maka model antriannya adalah (M/M/3)
lambda_val = λ
mu_val = μ
s_val = 3

hasil = hitung_mms(lambda_val, mu_val, s_val)
```

Fungsi **hitung_mms** dijalankan dengan memasukkan parameter λ , μ , dan s sebagai input untuk mensimulasikan kinerja sistem antrian M/M/3.

2. Output Model Antrian M/M/3

```
=== HASIL SIMULASI MODEL ANTRIAN M/M/3 ===
Jumlah server (s): 3
Laju kedatangan (λ): 131.56 pelanggan/jam
Laju pelayanan (μ): 54.89 pelanggan/jam per server
Tingkat kesibukan sistem (ρ): 0.7989
➡ Sistem dalam kondisi efisien (stabil dan optimal).

=== Parameter Kinerja Sistem ===
Probabilitas tidak ada pelanggan (P0): 0.0566
Rata-rata pelanggan dalam antrian (Lq): 3.0000
Rata-rata pelanggan total dalam sistem (L): 5.0000
Waktu tunggu rata-rata dalam antrian (Wq): 1.17 menit
Waktu rata-rata dalam sistem (W): 2.26 menit
```

=== ANALISIS BIAYA ===

```
Biaya pelayanan per jam : Rp90,000
Biaya menunggu per jam  : Rp25,644
Total biaya per jam      : Rp115,644
```



Interpretasi:

```
Sistem berada pada kondisi efisien (stabil dan seimbang).
🕒 Waktu tunggu rata-rata sangat baik (<5 menit).
```

Simulasi model M/M/3 menunjukkan bahwa sistem pelayanan berada pada kondisi seimbang dengan waktu tunggu rendah dan biaya operasional yang efisien

Simulasi M/M/4

Hasil Simulasi 4 Loker Pembayaran



1. Input Parameter & Pemanggilan Fungsi

```
[8]: # Jika terdapat 4 loket, maka model antrian nya adalah (M/M/4)
      lambda_val = λ
      mu_val = μ
      s_val = 4

      hasil = hitung_mms(lambda_val, mu_val, s_val)
```

Fungsi **hitung_mms** dijalankan dengan memasukkan parameter λ , μ , dan s sebagai input untuk mensimulasikan kinerja sistem antrian M/M/4.

2. Output Model Antrian M/M/4

```
=== HASIL SIMULASI MODEL ANTRIAN M/M/4 ===
Jumlah server (s): 4
Laju kedatangan (λ): 131.56 pelanggan/jam
Laju pelayanan (μ): 54.89 pelanggan/jam per server
Tingkat kesibukan sistem (ρ): 0.5992
➡ Sistem cukup longgar, petugas cenderung sering idle.
```

```
=== Parameter Kinerja Sistem ===
Probabilitas tidak ada pelanggan (P0): 0.0834
Rata-rata pelanggan dalam antrian (Lq): 0.4276
Rata-rata pelanggan total dalam sistem (L): 2.8243
Waktu tunggu rata-rata dalam antrian (Wq): 0.20 menit
Waktu rata-rata dalam sistem (W): 1.29 menit
```

```
=== ANALISIS BIAYA ===
Biaya pelayanan per jam : Rp120,000
Biaya menunggu per jam : Rp4,276
Total biaya per jam : Rp124,276
```

 Interpretasi:
Sistem tergolong longgar, petugas sering idle dan biaya relatif tinggi.
 Waktu tunggu rata-rata sangat baik (<5 menit).

Simulasi model M/M/4 menunjukkan bahwa sistem pelayanan berada pada kondisi longgar dengan waktu tunggu sangat singkat dan biaya operasional yang efisien

Perbandingan Hasil Simulasi Tiap Locket

Hasil	M/M/1	M/M/2	M/M/3	M/M/4
P0	0	0	0.0567	0.0834
Lq (Satuan Orang)	77	22	3	4
Lq (Satuan Orang)	79	24	5	3
Wq (Menit)	34.97	9.93	1.17	0.2
W (Menit)	96.06	11.03	2.26	1.29
Cs	Rp30.000	Rp60.000	Rp90.000	Rp120.000
Cw	Rp766.667	Rp217.778	Rp25.664	Rp4.276

Kesimpulan



Proyek menunjukkan bahwa konfigurasi M/M/3 paling optimal untuk operasi loket registrasi harian karena menyeimbangkan kinerja antrian dan biaya pelayanan. Tiga loket menjaga utilisasi pada rentang aman, meningkatkan peluang sistem kosong, serta menurunkan panjang antrian dan waktu tunggu sehingga penumpukan pada jam normal terkendali.

M/M/4 dipertimbangkan secara situasional pada periode sangat sibuk seperti awal semester atau batas akhir administrasi, saat target waktu tunggu sangat ketat, atau ketika kapasitas tiga loket menurun. Di luar kondisi tersebut, sistem menjadi terlalu longgar, petugas cenderung idle, dan biaya relatif lebih tinggi.

Thank You!



Kelompok 4

